Longest Palindromic Substring

OCT 02, 2023 #DSA #LEETCODE #STRINGS

The problem of finding the Longest Palindromic Substring is a classic and fundamental challenge in computer science and string manipulation. A palindrome is a sequence of characters that reads the same forwards as it does backward. In this problem, the task is to identify and extract the longest substring within a given input string that forms a palindrome.

Several common methods and algorithms exist to solve this problem, each with its own trade-offs in terms of time and space complexity. Here is a brief introduction to some of the common methods:

- **Brute Force**: The simplest approach involves checking all possible substrings in the input string to see if they are palindromes. While conceptually straightforward, this method has a time complexity of O(n^3) and is not practical for large input strings.
- **Expand Around Center**: The approach treats each character in the input string as a potential center of a palindrome and expands outward to find the longest palindrome. It has a time complexity of O(n^2) and is often faster in practice due to small constant factors.
- **Dynamic Programming**: This approach utilizes a dynamic programming table to store information about whether substrings are palindromes or not. It has a time complexity of O(n^2) and is more efficient than brute force.
- **Manacher's Algorithm**: This is an advanced and highly efficient algorithm that can find the longest palindromic substring in linear time, O(n). It employs clever techniques to avoid redundant character comparisons.
- **Suffix Tree**: Constructing the suffix tree takes O(n^2) time, but once built, it can answer palindrome-related queries efficiently.
- **Palindrome Tree**: A specialized data structure known as a palindrome tree is designed for solving palindrome-related problems efficiently. It can find the longest palindromic substring in linear time.

During coding interview, it's generally not recommended to start with the brute force solution. Dynamic programming is a good choice for an interview because it's a well-known technique with a reasonable time complexity O(n^2). Expand around center is also a good choice for a coding interview because it's relatively easy to implement.

While highly efficient, Manacher's algorithm, Suffix Tree or Palindrome Tree are typically not suitable for a coding interview unless the interviewer explicitly asks for them. They are complex to implement and may not be expected in most interview scenarios.

Expand Around Center

The idea behind this approach is to consider each character in the given string as a potential center of a palindrome and then expand outward to check if the characters on both sides form a valid palindrome.

This technique is effective for solving problems where you need to find palindromes because it takes advantage of the inherent symmetry of palindromes.

```cpp
#include <cassert>
#include <iostream>
#include <string>
using namespace std;

string expandAroundCenter(string s, int left, int right) {
  while (left >= 0 && right < s.length() && s[left] == s[right]) {
    left--;
    right++;
  }
  return s.substr(left + 1, right - left - 1);
}

string longestPalindrome(string s) {
  if (s.empty())
    return "";

  string longest = "";
  for (int i = 0; i < s.length(); i++) {
    string oddPalindrome = expandAroundCenter(s, i, i);
    string evenPalindrome = expandAroundCenter(s, i, i + 1);

    if (oddPalindrome.length() > longest.length()) {
      longest = oddPalindrome;
    }
    if (evenPalindrome.length() > longest.length()) {
      longest = evenPalindrome;
    }
  }

  return longest;
}

int main() {
  assert(longestPalindrome("a") == "a");
  assert(longestPalindrome("cbbd") == "bb");
  cout << "All tests passed!" << endl;
  return 0;
}
```

This algorithm has a time complexity of O(n^2) because for each character in the string, it potentially expands outward to both the left and right sides. Space complexity is O(1) as it only requires a few extra variables to keep track of the longest palindrome found so far and the current positions of the pointers as it iterates through the input string.

Dynamic Programming

The dynamic programming approach for finding the longest palindromic substring involves using a table to store whether substrings are palindromes or not. This approach has a time complexity of O(n^2), where n is the length of the input string.

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <cassert>
using namespace std;

string longestPalindrome(string s) {
  if (s.empty())
    return "";
  int n = s.length();
  vector<vector<bool>> dp(n, vector<bool>(n, false));
  int start = 0;         // Start index of the longest palindromic substring
  int max_length = 1;    // Length of the longest palindromic substring

  // All substrings of length 1 are palindromes
  for (int i = 0; i < n; i++) {
    dp[i][i] = true;
  }

  // Check for palindromes of length 2
  for (int i = 0; i < n - 1; i++) {
    if (s[i] == s[i + 1]) {
      dp[i][i + 1] = true;
      start = i;
      max_length = 2;
    }
  }

  // Check for palindromes of length 3 or more
  for (int len = 3; len <= n; len++) {
    for (int i = 0; i < n - len + 1; i++) {
      int j = i + len - 1; // Ending index of the current substring

      // Check if the current substring is a palindrome
      if (s[i] == s[j] && dp[i + 1][j - 1]) {
        dp[i][j] = true;

        // Update the longest palindrome information if needed
        if (len > max_length) {
          start = i;
          max_length = len;
        }
      }
    }
  }

  // Extract and return the longest palindromic substring
  return s.substr(start, max_length);
}

int main() {
  assert(longestPalindrome("a") == "a");
  assert(longestPalindrome("cbbd") == "bb");
  cout << "All tests passed!" << endl;
  return 0;
}
```


RELATED CONTENT

Polynomial Rolling Hash
Big-O Notation
Rabin–Karp String Searching
Dynamic Programming (DP)
Bloom Filter